

实验四 数据平面编程实践

一、 教学目的与学时建议

1. 能够在 linux 系统下安装、配置 P4 编程与运行实验环境；
2. 能够运用 P4 语言编写数据平面的二层与三层转发程序；
3. 能够调试程序和实现 P4 程序的运行。
4. 建议：4 学时

二、 技术与知识背景

1. P4 的编程模型

P4 是一种协议无关的数据包处理编程语言，P4 支持用户自定义匹配字段，协议解析过程和转发过程，从而能实现真正意义上的协议无关可编程网络数据平面。P4 与 openflow 的定位完全不同。openflow 是一种南向协议，它是控制平面与数据平面沟通的桥梁。而 P4 则是一门数据平面的编程语言，它关注的是数据平面的开放性，并不涉及到控制平面。

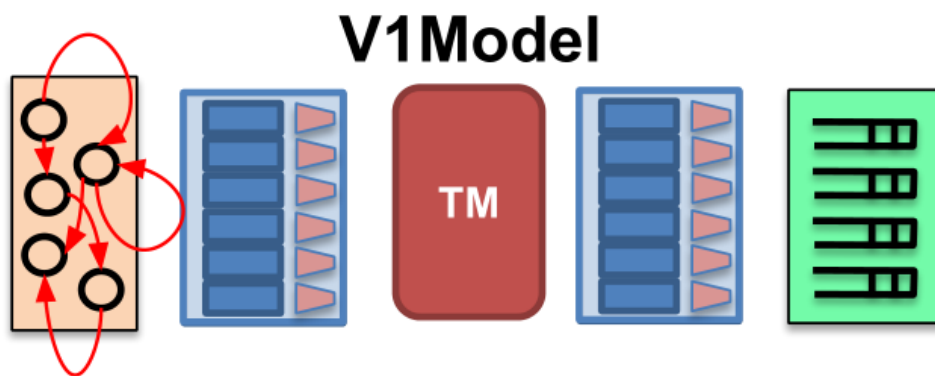


图 1 P4 的抽象转发模型 V1Model

P4 提供了最简单易理解的编程结构 V1Model，如图 1 所示，从图上可以看出，数据包的处理过程像一条流水线一样，它由 5 个模块组成，分别是： 1. Parser：解析器，对数据包根据预先定义好的格式进行解析，并提取头部的各个字段，在这里可能会根据不同的头部进行匹配。比如如果三层包头的协议字段是 0x0800 则是 ip 报文，如果是 0x0806 则是 arp 报文，则会进行不同的处理。 2. Ingress：Ingress 流水线部分，数据处理逻辑的入口。 3. TM：Traffic Manager，在这里定义核心的处理逻辑，并且会对一些元数据进行更新。 4. Egress：Egress 流水线部分，数据处理逻辑的出口。 5. Deparser：用于对数据包进行重组，因为数据包在处理过程中经历了分解和处理。所以最后转发的时候需要重组一下。

2. P4 代码结构

P4 为我们提供了上述抽象，我们就可以把所有的交换机的内部处理逻辑理解为上述的模型，每次数据包进入交换机，都会通过这些一条流水线。你要做的只是去定义流水线上的各个模块罢了。所以，按照上述模型，P4 语言的代码结构如下：

```
#include <core.p4>

#include <v1model.p4>

const bit<16> TYPE_IPV4 = 0x800;

/*****

***** HEADERS *****

*****/

typedef bit<9>  egressSpec_t;

typedef bit<48> macAddr_t;

typedef bit<32> ip4Addr_t;

/*定义二层数据包头*/

header ethernet_t {

    macAddr_t dstAddr;

    macAddr_t srcAddr;

    bit<16>  etherType;

}

/*定义 ip 数据包头*/

header ipv4_t {

    bit<4>    version;

    bit<4>    ihl;

    bit<8>    diffserv;

    bit<16>   totalLen;

    bit<16>   identification;

    bit<3>    flags;

    bit<13>   fragOffset;
```

```

        bit<8>    ttl;

        bit<8>    protocol;

        bit<16>   hdrChecksum;

        ip4Addr_t srcAddr;

        ip4Addr_t dstAddr;

    }

    struct metadata {

        /* empty */

    }

    /*定义了 headers 由二层头部与三层头部组成*/

    struct headers {

        ethernet_t    ethernet;

        ipv4_t        ipv4;

    }

    /*****

    ***** P A R S E R *****

    *****/

    parser MyParser(packet_in packet,

                    out headers hdr,

                    inout metadata meta,

                    inout standard_metadata_t standard_metadata) {

        state start {

            transition parse_ethernet; //转移到 parse_ethernet 状态

        }

        state parse_ethernet {

            packet.extract(hdr.ethernet); //提取数据包头

            transition select(hdr.ethernet.etherType) {

                TYPE_IPV4: parse_ipv4; //转移到 parse_ipv4 状态

                default: accept;

```

```

    }
}

state parse_ipv4 {

    packet.extract(hdr.ipv4);

    transition accept; //根据 etherType, 转移到其他状态, 直到转移到 accept 结束

}

}

```

```

/*****

```

```

*****      CHECKSUM      VERIFICATION      *****

```

```

*****/

```

```

/*校验和的计算可以空着*/

```

```

control MyVerifyChecksum(inout headers hdr, inout metadata meta) {

```

```

    apply { }

```

```

}

```

```

/*****

```

```

*****      INGRESS      PROCESSING      *****

```

```

*****/

```

```

control MyIngress(inout headers hdr,

```

```

    inout metadata meta,

```

```

    inout standard_metadata_t standard_metadata) {

```

```

/*定义了丢包的动作*/

```

```

action drop() {

```

```

    mark_to_drop();

```

```

}

```

```

/*定义了三层转发的动作*/

```

```

action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {

```

```

    standard_metadata.egress_spec = port;

```

```

        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;

        hdr.ethernet.dstAddr = dstAddr;

        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
    }

    /*定义 table ipv4_lpm*/
    table ipv4_lpm {
        key = {
            hdr.ipv4.dstAddr: lpm;
        }
        actions = {
            ipv4_forward;
            drop;
            NoAction;
        }
        size = 1024;
        default_action = drop();
    }

    //上述都是声明定义过程，这里是真正的数据处理逻辑
    apply {
        if (hdr.ipv4.isValid()) {
            ipv4_lpm.apply();
        }
    }
}

/*****

***** EGRESS PROCESSING *****

*****/

```

```

control MyEgress(inout headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {
    apply { }
}

/*****
*****      CHECKSUM      COMPUTATION      *****
*****/

/*校验和的计算部分，可以省略*/

control MyComputeChecksum(inout headers  hdr, inout metadata meta) {
    apply {
        update_checksum(
            hdr.ipv4.isValid(),
            { hdr.ipv4.version,
            hdr.ipv4.ihl,
            hdr.ipv4.diffserv,
            hdr.ipv4.totalLen,
            hdr.ipv4.identification,
            hdr.ipv4.flags,
            hdr.ipv4.fragOffset,
            hdr.ipv4.ttl,
            hdr.ipv4.protocol,
            hdr.ipv4.srcAddr,
            hdr.ipv4.dstAddr },
            hdr.ipv4.hdrChecksum,
            HashAlgorithm.csum16);
    }
}

```

```

/*****

*****  D E P A R S E R  *****

*****/

/*数据包的重组阶段*/

control MyDeparser(packet_out packet, in headers hdr) {

    apply {

        packet.emit(hdr.ethernet);

        packet.emit(hdr.ipv4);

    }

}

/*****

*****  S W I T C H  *****

*****/

V1Switch(

MyParser(),

MyVerifyChecksum(),

MyIngress(),

MyEgress(),

MyComputeChecksum(),

MyDeparser()

) main;

```

这个简单的 P4 语言程序定义了一个三层数据包的处理流程。该程序的主函数入口在最下方的 main 函数，在这里定义了一条完整的数据处理流水线。先是定义 Parser，后计算校验和，再经过 Ingress 和 Egress 对数据包进行处理，之后计算校验和，再将数据包重组就完成了了一次数据转发。

P4 程序的核心部分是数据头部的定义以及 Ingress 中对数据包的处理。在数据头部的定义中，可以很明确看到定义了二层以太网包头以及 IP 包头这两种数据格式。当然如果你需要实现更加复杂的应用，你可以定义更多的数据包头例如 MPLS 包头，TCP 包头等，甚至

你也可以定义 openflow 的包头，从而在 P4 程序中实现 openflow 的数据平面。

Ingress 中的数据包处理是 P4 的核心部分。示例程序中定义了表 `ipv4_lpm`。在 table 中可以定义动作，`ipv4_lpm` 表中的动作包括三层转发(`ipv4_forward`)，丢包(`drop`)和 `NoAction`，默认动作则是简单的丢包。`ipv4_forward` 这个 action 的实现逻辑也非常清晰，首先修改元数据 (metadata)，将 egress 的端口设置为出端口。然后，将二层的源 MAC 地址修改为当前 MAC 地址，将目的 MAC 修改为需要发送出去的端口的 MAC 地址，之后将 ttl 减一，这个逻辑也与我们所知道的三层转发过程一致。

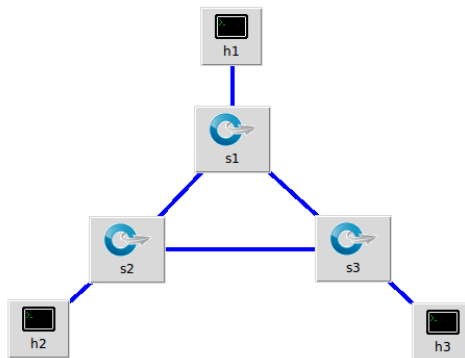
3. P4 学习资源

P4 官方的示例教程，地址是 <https://github.com/p4lang/tutorials>。里面的应用包括了基于 P4 实现的 tunnel、负载均衡、源路由、ecn 等等

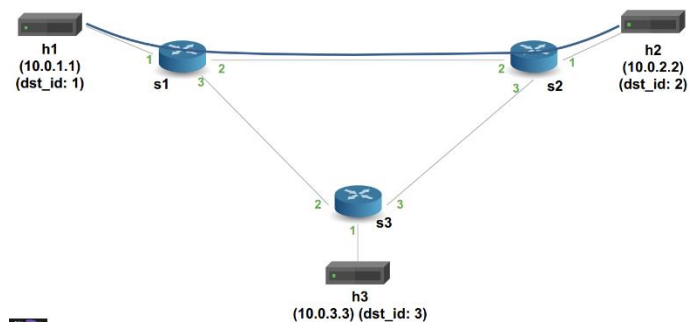
三、 基础技能

1. 任务要求

- 1) 在虚拟机中导入镜像 P4-Suite2018.ova。（下载链接：<https://pan.baidu.com/s/1gkbWIHLG43hFMLWWS9f97w>，提取码: egwf);
- 2) 使用 mininet 构建如下 SDN 网络拓扑。



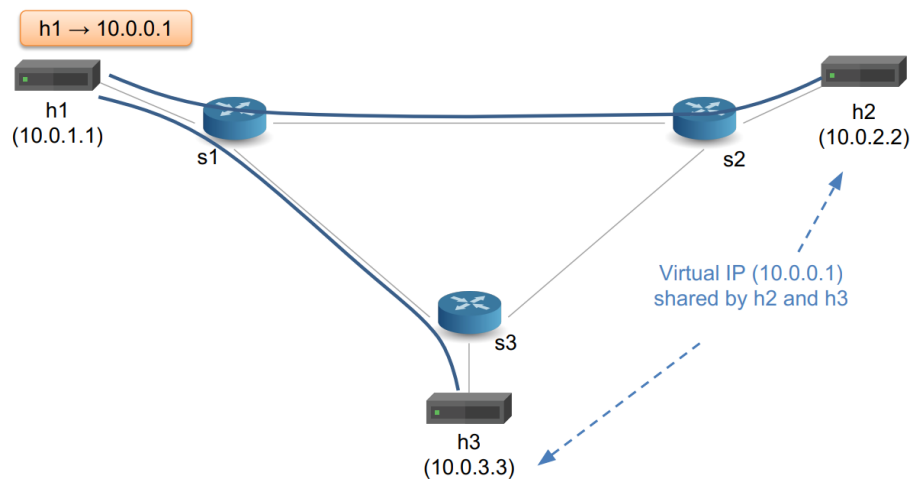
- 3) 在该环境中利用 P4 编写程序实现简单的 SDN 交换机转发功能。
- 4) 调试 P4 程序，进行运行与测试。
- 5) 为前述拓扑中的交换机增加基本的隧道协议支持功能。使发送方不仅仅可以通过目标 IP 地址发送数据包，还可以通过隧道号指定发送目标。比如在如下图中，通过携带参数 `dst_id` 为 2，指定包发送目标为 h2 交换机。（提示：参考 P4 tutorial 中 exercise 里的 basic tunnel 示例，为 IP 包增加一个封装头，在封装头中包含隧道协议号和目标隧道 ID）



四、进阶技能

1. 任务要求

- 1) 在基本技能的拓扑中为交换机完成简单的负载均衡功能，参考 exercise 中的 `load_balalce` 的例子,实现如下图的功能:通过虚拟 IP 地址 10.0.0.1, 程序能将数据包随机发给 h1 或者 h2 主机。



- 2) 在基本技能的拓扑中为交换机完成简单的源路由功能，参考 exercise 中的 `source_routing` 的例子,实现如下图的功能:程序能通过输入 2 3 2 2 1, 指定 h1 发往 h2 的数据包必须途径 s1, s2, s3, s1, s2 后才能达到。

